

What is Deadlock?

- Deadlock happens when a group of processes are stuck because they are waiting for resources that other processes are holding.

Processes Waiting for Each Other:

- Each process in a deadlock situation is waiting for an event that can only happen when another process, which is also waiting, triggers it.

Why is it Permanent?

- Deadlock is permanent because no process can move forward. Each process is waiting for another to finish, but since no event happens, the processes in the system cannot move forward at all.

Hard to Solve:

- There is no simple or efficient solution to deadlock in general, making it a challenging problem to fix in systems.

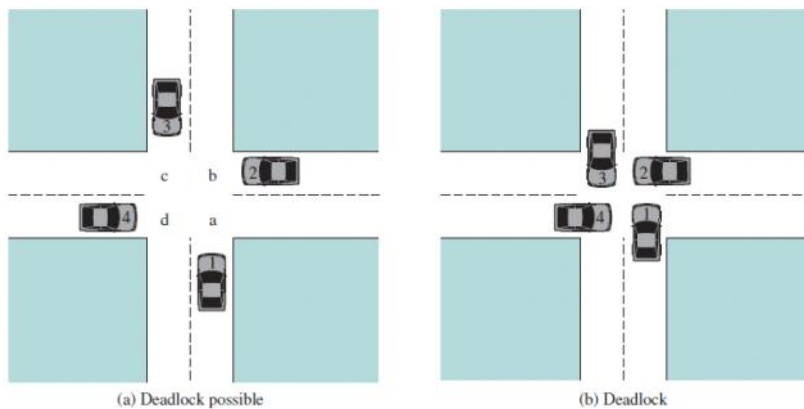


Figure 6.1 Illustration of Deadlock

RESOURCE CATEGORIES

Reusable

- Can be safely used by only one process at a time and is not finished by that use
- EX : Processors, I/O channels, main and secondary memory, devices, and data structures such as files, databases, and semaphores

Consumable

- One that can be created (produced) and destroyed (consumed)
- Interrupts, signals, messages, and information in I/O buffers.

Deadlock Approaches

There is no **one** effective strategy that can deal with all types of deadlock. There're three approaches are common :

- Deadlock Prevention**

stop one of the three conditions needed for deadlock to happen, or to **prevent circular waiting between processes**.

- Deadlock Avoidance**

Do not allow a resource request if this allocation might lead to deadlock.

- Deadlock Detection**

Allow resource requests when possible, but periodically **check** for the presence of deadlock and take action to recover.

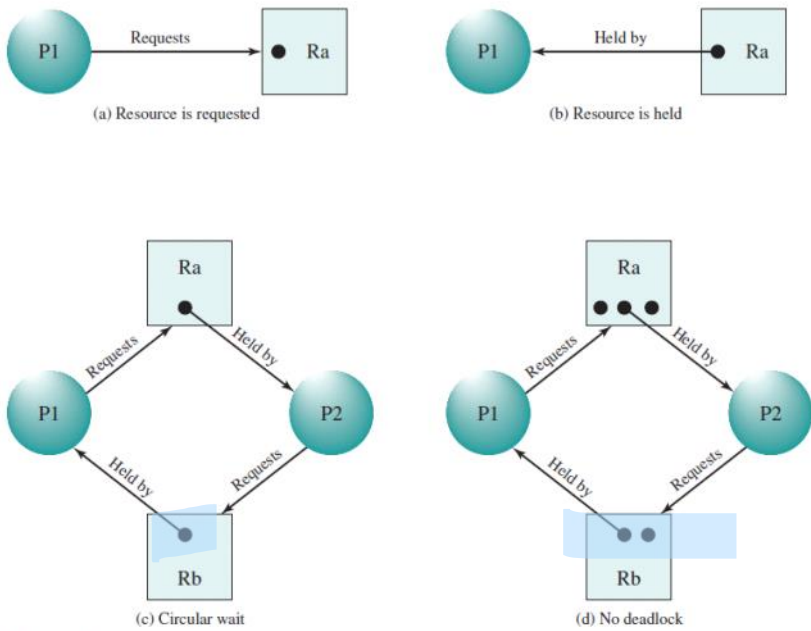


Figure 6.5 Examples of Resource Allocation Graphs

6.5c shows an example deadlock. There is only one unit each of resources Ra and Rb. Process P1 holds Rb and requests Ra, while P2 holds Ra but requests Rb. Figure 6.5d has the same topology as Figure 6.5c , but there is no deadlock because multiple units of each resource are available.

CONDITIONS FOR DEADLOCK

Mutual Exclusion

- Only one process may use a resource at a time.
- No process may access a resource unit that has been allocated to another process.

Hold-Wait

- A process can keep the resources it already has while it waits for other resources.

No pre-emption(ön-alım hakkı yok)

- No resource can be forcibly removed from a process holding it.

Circular Wait

Circular wait means that a **group of processes are waiting for each other in a loop**. Each process holds a resource that the next process in the group needs. For example, Process A waits for a resource from Process B, Process B waits for a resource from Process C, and Process C waits for a resource from Process A. This creates a cycle where no process can move forward, leading to a deadlock.

The fourth condition is, actually, a potential consequence of the first three.

Deadlock Prevention Strategy

- Design a system in such a way that the possibility of deadlock is not included.

Two main method :

Indirect : The indirect method works by stopping at least one of (**Mutual Exclusion, Hold and Wait, No Preemption**) these conditions from happening.

Direct: Focus on preventing **circular wait**.

Deadlock Condition Prevention

Mutual Exclusion

- If a resource needs mutual exclusion, the operating system must support it.
- Deadlock can happen if more than one process needs to write to the resource at the same time.

Hold and Wait

- To prevent deadlock, a process must request all needed resources at once.
- Process will wait until all resources are available.

No Preemption

- If a process can't get a new resource, it must give up its current resources and try again.
- The operating system can also stop another process and take its resources if needed.

Circular Wait

- The circular wait can be prevented by defining a linear ordering of resource type.

Deadlock Avoidance (Deadlock'tan Kaçınma)

- Allows the three conditions for deadlock but makes careful about deadlock point is never reached.
- Decides dynamically if a resource request could lead to deadlock before confirm it.
- Requires knowledge of future resource requests to make decisions.

Two Approaches to Deadlock Avoidance (Deadlock'tan Kaçınma)

1- Resource Allocation Denial (Kaynak Tahsisinin Reddi)

- Don't give a process extra resources if this could cause a deadlock.

2- Process Initiation Denial (Process Başlatma Reddi)

- Don't start a process if its needs could cause a deadlock.

Resource Allocation Denial

- referred as the *banker's algorithm*.
- **The state** of the system shows how resources are currently allocated to processes.
- **Safe state** is when resources can be allocated without causing deadlock.
- **Unsafe state** is any state where deadlock could happen.

Deadlock Avoidance Advantages (Deadlock'tan Kaçınmanın Avantajları)

- That it is not necessary to preempt and rollback processes, like deadlock detection.
- It is less strict(katı) than deadlock prevention.

Deadlock Avoidance Restrictions (Deadlock'tan Kaçınmanın Getirdiği Kısıtlamalar)

- Each process's maximum resource need must be known in before.
- The processes must be independent.
- There must be a fixed number of resources to allocate.
- No process can exit while holding resources.

Deadlock Strategies

- Deadlock prevention strategies are very **conservative**(tutucu).
--> Limit access to resources .
- Deadlock detection strategies do the opposite.
--> Resource request **are given whenever they can be approved.**

Deadline Detection Algorithm

Deadlock can be checked every time a resource is requested or less often.

Advantages

- It leads to early detection.
- Algorithm is very simple.

Disadvantages

- frequent check consume serious processor time

Recovery Strategies ("Kurtarma Stratejileri" deadlock için)

- **Cancel all processes** that are deadlocked.
- **Backup each deadlocked process** to a previous point and restart all processes
- Cancel deadlocked processes **one by one** until there is no deadlock.
- **Preempt resources one by one** until deadlock is removed. (Deadlock (kilitlenme) ortadan kalkana kadar kaynakları birer birer elinden al.

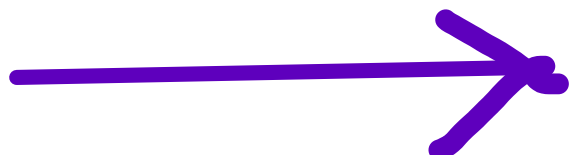
Integrated Deadlock Strategy

There're different strategies in different situations for efficientness.

- Group resources into several resource classes.
- Arrange resources in order to prevent deadlock between different types
- **For each resource class, use the most suitable algorithm.**

Classes of resources :

- **Swappable space: Blocks of memory** on secondary storage used for swapping processes.
- **Process resources:** Devices like **tape drives** and **files** that can be assigned.
- **Main memory:** Assigned to processes in **pages** or **segments**.
- **Internal resources:** Resources like **I/O channels**.



Class Strategies

- **Swappable space:** Prevent deadlock by allocating all resources at once if storage needs are known.
- **Process resources:** Avoid deadlock by declaring resources in before and using resource ordering.
- **Main memory:** Use preemption by moving processes to secondary memory to prevent deadlock.
- **Internal resources:** Prevent deadlock with resource ordering(kaynak sıralaması).

Dining Philosophers Problem

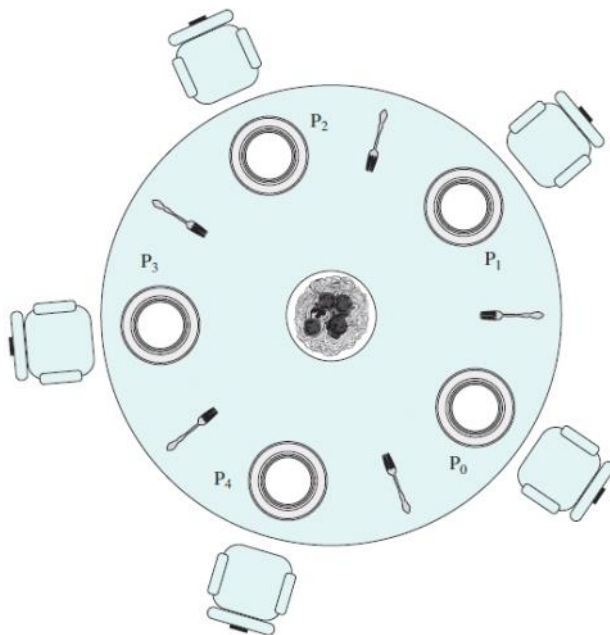


Figure 6.11 Dining Arrangement for Philosophers

- No two philosophers can use the same fork at the same time (mutual exclusion)
- No philosopher must starve to death (avoid deadlock and starvation)